



# Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

## pandas: Combining and Loading

WEEK 2 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# By the end of today you can

## Join tables on a shared column

The job VLOOKUP does, with the row count checked.

```
pd.merge(holdings, ratings, on='rating')
```

## Summarize by group

Counts, totals, and averages. In Excel, a pivot table.

```
book.groupby('bucket')['oas'].mean()
```

## Report a weighted average

Market-value-weighted OAS and duration.

```
book['oas'] * book['market_value']
```

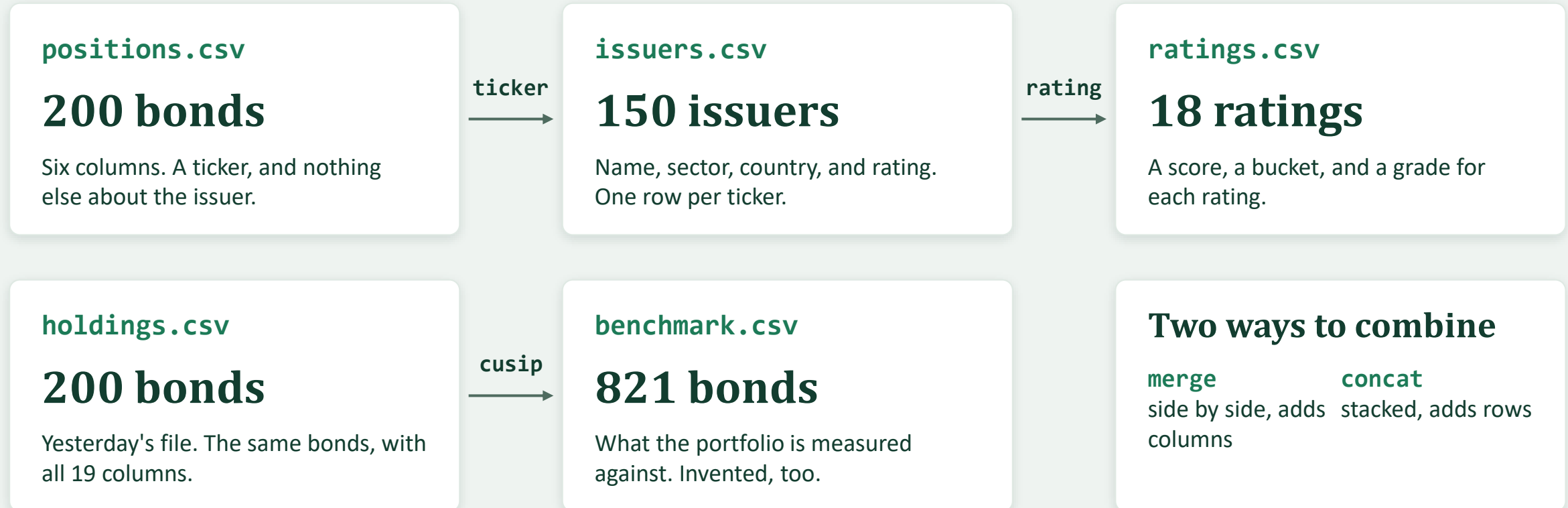
## Save results, work with dates

A CSV with no stray column. Date text as real dates.

```
pd.to_datetime(book['maturity'])
```

# Yesterday one table. Today, several.

Each file comes from a different system. A shared column, the key, links one to the next.



# merge: a lookup across tables

```
named = pd.merge(positions, issuer_details, on='ticker')
```

on= names the column to match on: the key. It appears once in the result.

positions: three of its six columns

| ticker | coupon | par_held |
|--------|--------|----------|
| BEZE   | 7.000  | 5000000  |
| BEZH   | 8.125  | 250000   |
| BEZL   | 5.125  | 1500000  |

+

issuer\_details: one row per ticker

| ticker | issuer             | sector         | rating |
|--------|--------------------|----------------|--------|
| BEZE   | Bezane Finance     | Financials     | BBB-   |
| BEZH   | Bezamyth Machinery | Industrials    | B      |
| BEZL   | Bezoquil Freight   | Transportation | BB     |

## Check the count

200 rows in, 200 rows out.  
Six columns became nine.

named: every position now carries its issuer, its sector, and its rating

| ticker | coupon | par_held | issuer             | sector         | rating |
|--------|--------|----------|--------------------|----------------|--------|
| BEZE   | 7.000  | 5000000  | Bezane Finance     | Financials     | BBB-   |
| BEZH   | 8.125  | 250000   | Bezamyth Machinery | Industrials    | B      |
| BEZL   | 5.125  | 1500000  | Bezoquil Freight   | Transportation | BB     |

## In Excel

**=VLOOKUP(B2, issuers!A:C, 3, FALSE)**

Filled down 200 rows, once for each column wanted.

# how= decides which rows are kept

desk\_small

| ticker | face    |
|--------|---------|
| QVNE   | 1500000 |
| DNMR   | 500000  |
| PLWK   | 750000  |
| HLVF   | 250000  |

sector\_small

| ticker | sector         |
|--------|----------------|
| QVNE   | Energy         |
| DNMR   | Transportation |
| PLWK   | Materials      |
| THAR   | Materials      |

```
left_join = pd.merge(
    desk_small, sector_small,
    on='ticker', how='left')
    ticker    face    sector
0  QVNE  1500000    Energy
1  DNMR   500000 Transportation
2  PLWK   750000    Materials
3  HLVF   250000         NaN
```

## how='inner'

Keys found in both tables. The default.

QVNE DNMR PLWK HLVF THAR

**3 rows** HLVF and THAR dropped

## how='left'

Every row of the left table. Like a VLOOKUP.

QVNE DNMR PLWK HLVF THAR

**4 rows** HLVF kept, its sector is NaN

## how='right'

Every row of the right table.

QVNE DNMR PLWK HLVF THAR

**4 rows** THAR kept, its face is NaN

## how='outer'

Every key from either table.

QVNE DNMR PLWK HLVF THAR

**5 rows** Nothing dropped

# A merge can add rows, and no error says so

desk\_small: 4 rows

| ticker | face    |
|--------|---------|
| QVNE   | 1500000 |
| DNMR   | 500000  |
| PLWK   | 750000  |
| HLVF   | 250000  |

+

sector\_twice: QVNE is listed twice

| ticker | sector         |
|--------|----------------|
| QVNE   | Energy         |
| QVNE   | Energy         |
| DNMR   | Transportation |
| PLWK   | Materials      |
| HLVF   | Consumer       |



doubled: 5 rows from a left merge

| ticker | face    | sector         |
|--------|---------|----------------|
| QVNE   | 1500000 | Energy         |
| QVNE   | 1500000 | Energy         |
| DNMR   | 500000  | Transportation |
| PLWK   | 750000  | Materials      |
| HLVF   | 250000  | Consumer       |

Face before the merge

**3,000,000**

4 rows in the desk's table.



Face after the merge

**4,500,000**

5 rows. QVNE is counted twice.

## The habit

Print the row count before and after every merge.

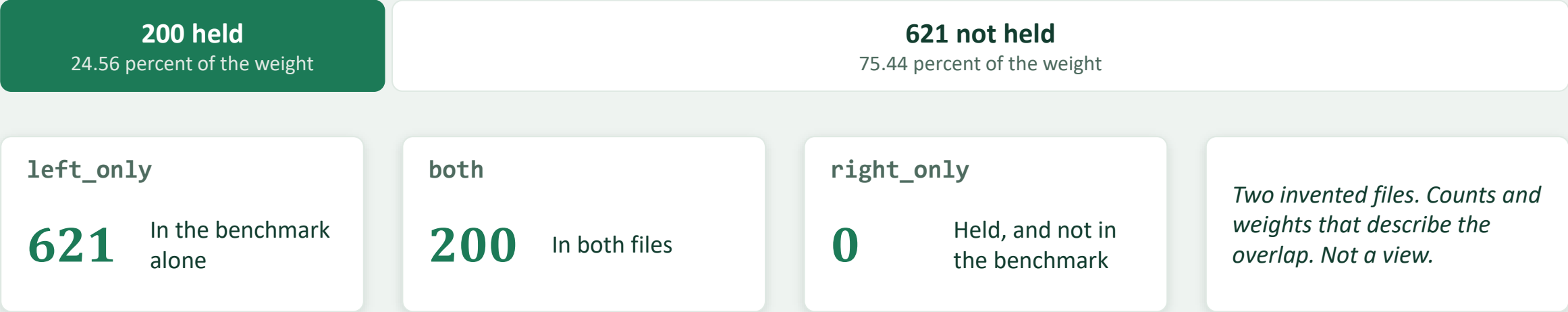
The cure: a lookup table with each key once. Check it with `nunique()`.

# The portfolio against the benchmark, as counts

```
desk_face = holdings[['cusip', 'par_held']]
compared = pd.merge(benchmark, desk_face,
                    on='cusip', how='left')
print('Rows:', compared.shape[0])
Rows: 821
```

| cusip     | ticker | weight_pct | par_held  |
|-----------|--------|------------|-----------|
| 99001BAA4 | QVNE   | 0.2076     | 2750000.0 |
| 99001BAB2 | QVNE   | 0.1551     | NaN       |
| 99001BAC0 | QVNE   | 0.0245     | NaN       |
| 99001BAD8 | QVNE   | 0.2024     | NaN       |

The benchmark's 821 bonds. Each box is as wide as its share of the benchmark's weight.



# concat stacks tables and adds rows

Two tables with the same 22 columns

**ig\_book**  
128 rows

**hy\_book**  
72 rows



**stacked**  
200 rows, 22 columns  
Market value 490,819,215

```
stacked = pd.concat([ig_book, hy_book])  
print('Stacked:', stacked.shape)  
Stacked: (200, 22)
```

Row labels where the first table ends and the second begins

As stacked

198

199

1

2

**ignore\_index=True**

126

127

128

129

## merge

Side by side. Adds columns. Matches the rows on a key.

## concat

Stacked. Adds rows. Lines the columns up by name.

## In Excel

Paste one sheet under another, or =VSTACK(). Excel pastes by position.

# groupby: split, apply, combine

`book.groupby('sector')`  
How to split the rows

`['market_value']`  
The column to use

`.sum()`  
The calculation

In Excel, a **pivot table**: sector in Rows, Sum of market\_value in Values.



| The same pattern                  | pandas                                                                                   |
|-----------------------------------|------------------------------------------------------------------------------------------|
| A mean, by rating bucket          | <code>book.groupby('bucket')['oas'].mean()</code>                                        |
| Several statistics at once        | <code>book.groupby('grade')['oas'].agg(['count', 'mean', 'median', 'min', 'max'])</code> |
| A summary table, column by column | <code>book.groupby('sector').agg(bonds=('cusip', 'count'), ...)</code>                   |

# A typical bond, or a typical dollar

## Simple average OAS

185.75

Every bond counts the same.

```
book['oas'].mean()
```

=AVERAGE(M2:M201)

## Weighted average OAS

180.46

Each bond counts in proportion to its market value.

```
(book['oas'] * book['market_value']).sum()  
/ book['market_value'].sum()
```

=SUMPRODUCT(M2:M201, S2:S201) / SUM(S2:S201)

**Technology: 298.2 simple, 228.2 weighted. The three widest of its 12 bonds explain the gap.**

Their share of the count



3 of 12 bonds: a quarter

Their share of market value



13.5 percent

| ticker | rating | oas  |
|--------|--------|------|
| FESR   | CCC+   | 1551 |
| FESR   | CCC+   | 614  |
| LORE   | BB-    | 334  |

*Neither figure is wrong. They answer different questions, about an invented portfolio.*

# A pivot table, a rule, and a file

```
pd.pivot_table(book, values='market_value',
                index='sector', columns='grade',
                aggfunc='sum', margins=True)
```

| sector    | High Yield  | Investment Grade | All          |
|-----------|-------------|------------------|--------------|
| Materials | 38,747,115  | 32,954,502.5     | 71,701,617.5 |
| Telecom   | 27,796,585  | 28,051,685       | 55,848,270   |
| All       | 169,901,850 | 320,917,365      | 490,819,215  |

```
book['price_label'] = book['price'].apply(
    price_label)
Discount    113
Premium     87
```

## apply runs a rule on every value

One price in, one label out. In Excel, a nested IF filled down. For rules, not for arithmetic.

Saving: `to_csv()` writes a table to a file, and `read_csv()` reads it back

### `book.to_csv(file)`

25 columns saved, 26 read back. The row numbers return as a stray column.

### `index=False`

25 saved, 25 read back. Row numbers carry no information, so leave them out.

### `sector_summary.to_csv(file)`

A grouped table keeps its index. It holds the sector names.

# Dates arrive as text

As loaded: text. It cannot be subtracted.

```
0    2032-06-18
1    2029-04-17
2    2030-06-03
Name: maturity, dtype: str
```

pd.to\_datetime()

Converted: real dates

```
0    2032-06-18
1    2029-04-17
2    2030-06-03
Name: maturity, dtype: datetime64[us]
```

The parts, with .dt

```
year      2032
month      6
day_name() Friday
```

```
days_left = (book['maturity'] - as_of).dt.days
book['years_to_maturity'] = days_left / 365.25
```

Shortest  
**1.0**

Longest  
**29.9**

Average  
**9.85**

*Years to maturity as of 2026-09-30, by calendar days. An approximation, not the 30/360 count used to price these bonds.*

pd.cut() sorts the years into buckets. Each box is as wide as its share of the 200 bonds.

**0 to 3**  
28 bonds  
75,125,702.5

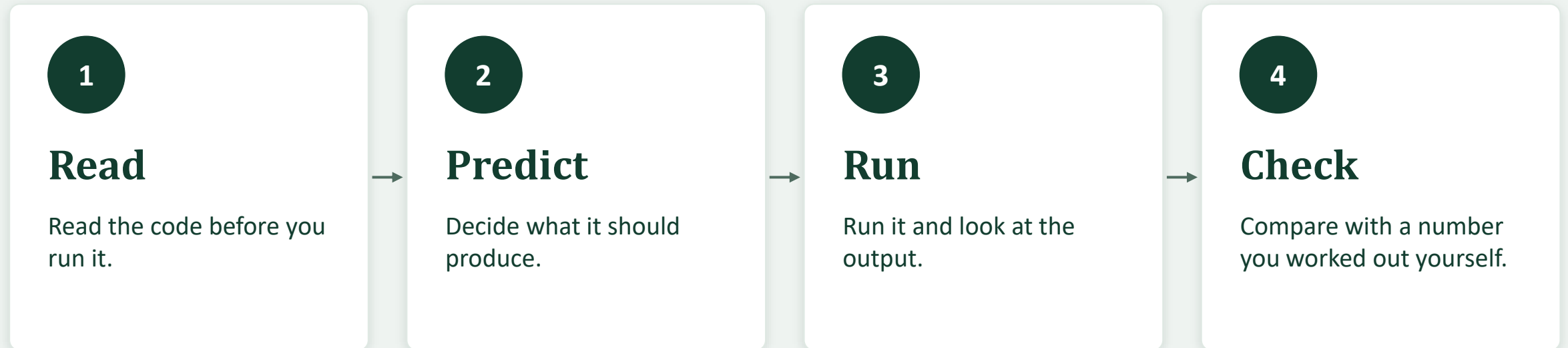
**3 to 7**  
51 bonds  
123,242,385

**7 to 15**  
88 bonds  
228,787,395

**15 and over**  
33 bonds  
63,663,732.5

# The AI check

Every notebook ends with code written in the style of an AI assistant. It has one bug. Find it.



*Code that runs is not the same as code that is right.*

# What you can do now

- Join positions to issuers and a rating scale, and check the row count every time
- Compare a portfolio with its benchmark by CUSIP
- Summarize by sector, rating bucket, or grade, and lay it out as a pivot table
- Report a market-value-weighted OAS or duration
- Save a result to a CSV file, and turn date text into a maturity profile

**Tomorrow** Case study: three tables from three systems, and a first look at each